

Analysis and Classification of the Forest Cover Types Dataset

University of Cagliari — coursework project, reworked

Samuele Nonnis

1 — Introduction

This report compares sixteen classification models on the Forest Cover Types dataset from scikit-learn. The task is to predict the forest cover type, and therefore the dominant tree species, for 30×30 m land cells located in the Roosevelt National Forest, Colorado, from cartographic features: elevation, slope, distances to water and roads, wilderness area and soil type. All 581,012 observations are used, with no subsampling.

The seven classes are strongly imbalanced, the largest holding 48.76% of the observations and the smallest 0.47%. Under this imbalance the aggregate metrics are not sufficient on their own. AdaBoost reaches an accuracy of 0.6688 and a macro-AUC of 0.8804 while its recall on classes 4, 5, 6 and 7 is 0.00, predicting only the three most frequent classes. The models are therefore ranked by weighted F1 with the macro F1 reported beside it, and every model is documented class by class.

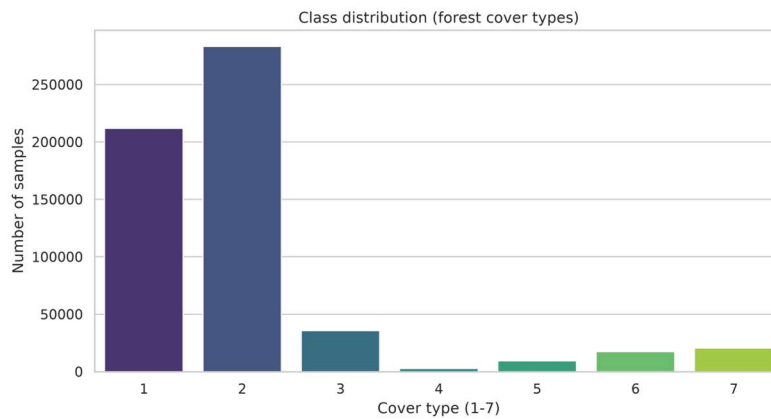
The protocol is identical for all sixteen models. Hyperparameters are searched on a stratified subset of 100,000 observations, the best estimator is refit on the full training set of 464,809 observations, and each model is scored once on the same held-out test set of 116,203 observations. Development ran locally and the final training in the cloud, to contain computation times; every number printed by the full run is recorded in `run_log_full.txt`.

2 — Dataset Exploration

2.1 Dataset Description

The dataset contains 581,012 observations and 7 classes representing the different tree species. The class distribution is heavily imbalanced: Spruce/Fir (1) and Lodgepole Pine (2) together make up 85.22% of the observations, while the remaining 14.78% is split among the other 5 classes. Cottonwood/Willow (4) and Aspen (5) are the rarest, at 0.47% and 1.63% of the total observations respectively. Because of this imbalance, the weighted F1-score is used as the reference metric.

Class	Name	Observations	%
1	Spruce/Fir	211,840	36.46%
2	Lodgepole Pine	283,301	48.76%
3	Ponderosa Pine	35,754	6.15%
4	Cottonwood/Willow	2,747	0.47%
5	Aspen	9,493	1.63%
6	Douglas Fir	17,367	2.99%
7	Krummholz	20,510	3.53%



The classification is based on 54 cartographic features:

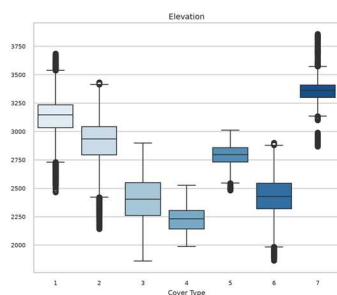
- 10 numerical;
- 4 binary features encoding the wilderness area in One-Hot Encoding;
- 40 binary features encoding the soil types in One-Hot Encoding.

Before the analysis, Reverse One-Hot Encoding was applied to collapse the 44 binary columns into 2 native categorical variables: **Wilderness_Area** and **Soil_Type**. To these, 3 engineered features were added, bringing the dataset to 15 features.

Type	Feature	Description
Numerical	Elevation	Elevation above sea level (m)
Numerical	Aspect	Aspect (°)
Numerical	Slope	Slope (°)
Numerical	Horiz. Dist. to Hydrology	Horizontal distance to water features (m)
Numerical	Vert. Dist. to Hydrology	Vertical distance to water features (m)
Numerical	Horiz. Dist. to Roadways	Distance to roadways (m)
Numerical (×3)	Hillshade 9am / Noon / 3pm	Hillshade indices at three times of day (0–255)
Numerical	Horiz. Dist. to Fire Points	Distance to fire ignition points (m)
Engineered	Distance_To_Hydrology	Euclidean distance to water features
Engineered (×2)	Elev_minus_VDH / Elev_plus_VDH	Combine elevation and vertical distance to hydrology
Categorical	Wilderness_Area	Protected natural area (4 categories)
Categorical	Soil_Type	Soil type (40 categories)

2.2 Numerical Feature Analysis

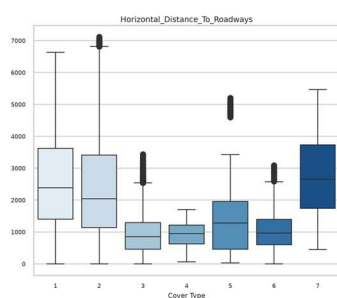
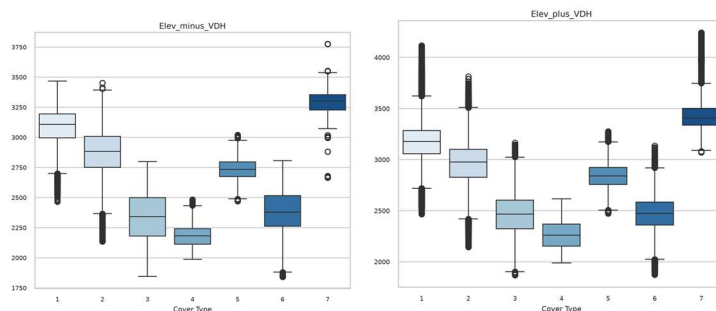
The dataset now includes 13 numerical features, analysed through boxplots and ordered by discriminative power across the 7 classes.



Elevation — elevation above sea level, in metres Elevation determines the temperature, humidity and climate conditions that define the ideal habitat for each species. It is useful for separating species adapted to harsh high-altitude climates from those found in the valleys. It is the most discriminative feature, with well-separated per-class distributions. Class (7) occupies the highest elevations (>3,300 m), with a compact distribution and many outliers towards higher elevations. Class (1) sits between 2,800–3,200 m, class (2) between 2,500–3,000

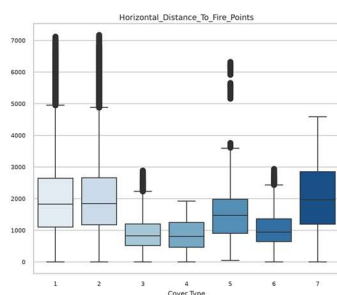
m. Classes 3, 4, 5, and 6 occupy the lower zones (<2,700 m) in a stratified manner, displaying few to no outliers.

Elev_minus_VDH and Elev_plus_VDH — engineered features The two features are linear combinations of Elevation and Vertical Distance to Hydrology, meant to capture the interactions between elevation and position relative to water features. Their distributions are driven by Elevation, with the same ordering and class separations, but creating these two features can help the tree-based models. In the model evaluation via Permutation Importance, Elev_minus_VDH turned out to be the most influential feature for the winning model.



Horizontal Distance to Roadways — distance to roadways This feature indicates the remoteness or accessibility of an area. For instance, mountain species are typically found farther from road networks. Classes 3, 4, 5, and 6 exhibit compact distributions with medians around 1,000 m. Conversely, classes 1, 2, and 7 display wider distributions, reaching distances up to 2,400 m, 2,000 m, and 2,700 m respectively. Consequently, this feature is highly discriminative, particularly for isolating classes 1, 2, and 7, with several classes showing

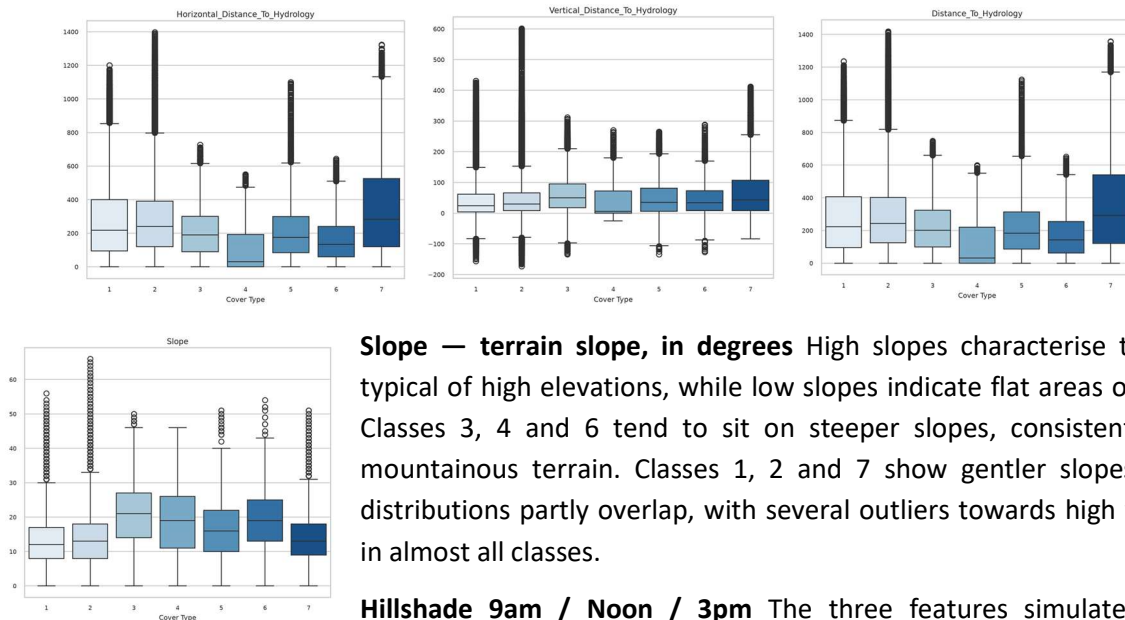
outliers at greater distances.



Horizontal Distance to Fire Points — distance to fire ignition points It measures the historical distance from areas hit by fires. Some species are more fire-resistant than others. None of the distributions exceed 3,000 m, outliers aside. Classes (1) and (2) show wide, almost identical distributions. Classes 3, 4, 5, 6 tend towards shorter distances and have compact distributions. Class (7) has no outliers. High-altitude species are therefore usually farther from fire-prone areas, while others grow near more fire-vulnerable areas.

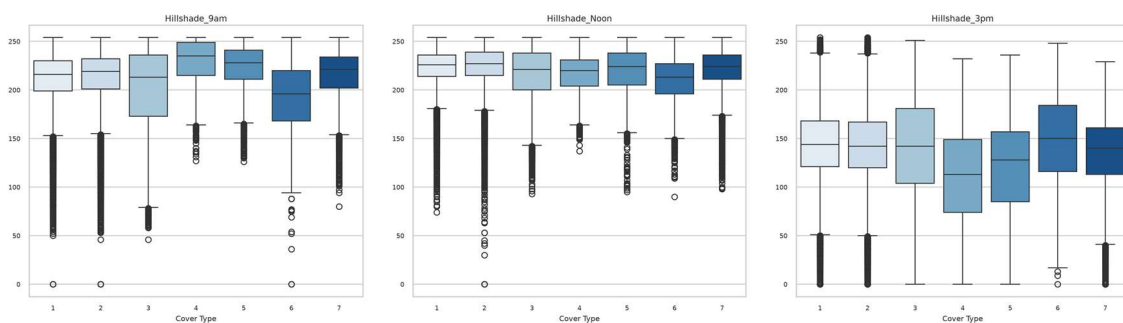
Horizontal / Vertical Distance to Hydrology and Distance to Hydrology — distances to water features

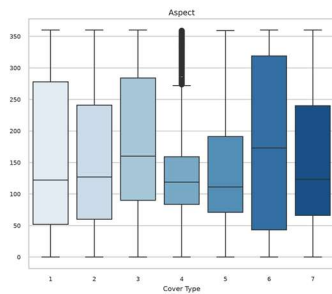
Horizontal, vertical and Euclidean distance to water features, used to identify species that need to live close to water. Class (4) is the most distinct in all three plots, with median distances around 0 m, very low compared to the other classes. The vertical distance shows low values not exceeding 50 m, with compact, overlapping distributions and many outliers across all classes. For the Euclidean and horizontal distances the distributions are more varied.



Slope — terrain slope, in degrees High slopes characterise terrain typical of high elevations, while low slopes indicate flat areas or hills. Classes 3, 4 and 6 tend to sit on steeper slopes, consistent with mountainous terrain. Classes 1, 2 and 7 show gentler slopes. The distributions partly overlap, with several outliers towards high values in almost all classes.

Hillshade 9am / Noon / 3pm The three features simulate solar exposure at three times of day (0–255) and act as indicators of exposure to the sun, which may affect the growth of species with different light tolerances. The distributions overlap heavily. For 9am they range over 170–250, with many outliers towards low values, and only class (6) seems to stand apart. For noon the values are very high, with compact, almost identical distributions and outliers towards low values. For 3pm the range is 70–180, with outliers in both directions. Overall, these features have weaker discriminative power than the other topographic variables.





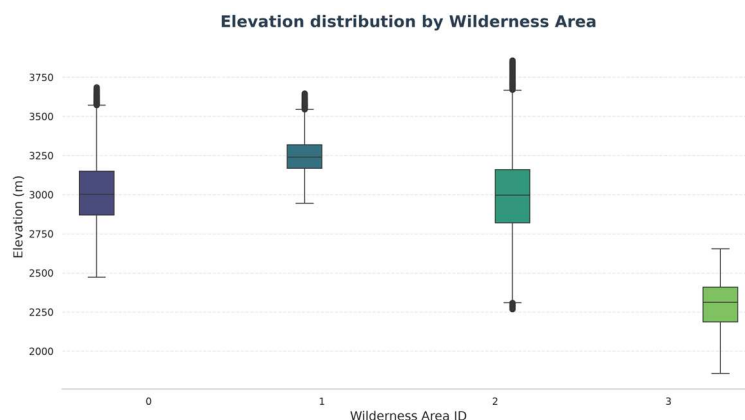
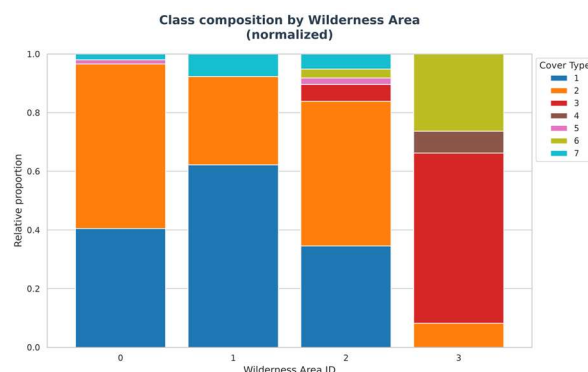
Aspect — slope orientation in degrees, 0–360° It indicates sun exposure and distinguishes whether a species prefers shadier or sunnier areas. The distributions overlap across most classes, so the feature has limited discriminative power. The medians fall between 110°–160°, except for class (6), which shows a higher value (~175°) with a wider distribution; this species is therefore presumed to occur on north-west or north-facing slopes. Only class 4 shows outliers towards high values. The feature is not an important predictor but can be useful

for class (6).

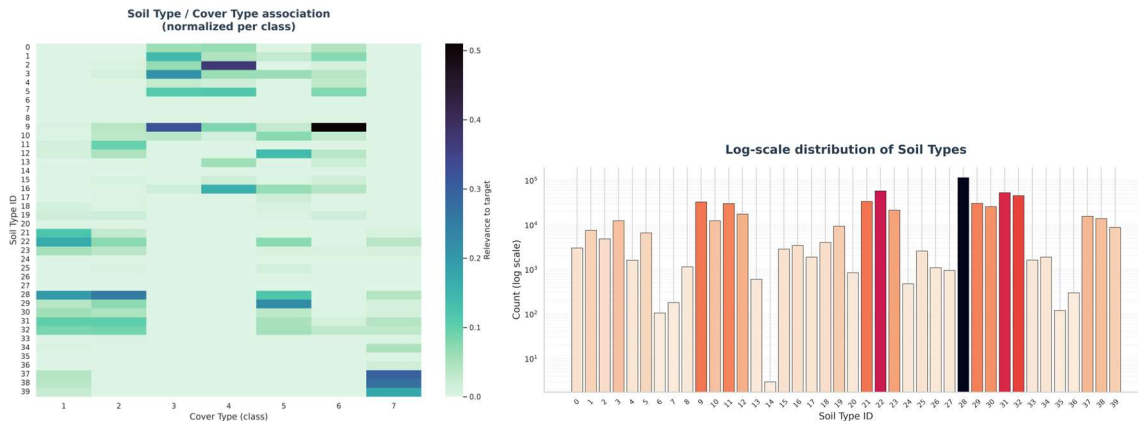
Summary. Elevation is the most discriminative feature, together with the engineered features correlated to it. Next, `Horizontal_Distance_To_Roadways` and `Horizontal_Distance_To_Fire_Points` contribute, and the hydrology distances are useful for isolating class (4). Hillshade and Aspect are the least informative on their own but can be useful in combination with other features.

2.3 Categorical Feature Analysis

Wilderness_Area — protected natural area It indicates which of four protected areas the tree species belongs to. This feature is a good predictor, especially when used together with elevation. In the class-composition plot, areas 0, 1 and 2 are dominated by classes (1) and (2). Area 3 sits at lower elevations, as the elevation-distribution boxplot shows. Area 3 is also the main habitat of classes (3), (6) and hosts almost all of class (4), which is nearly absent from the other areas.



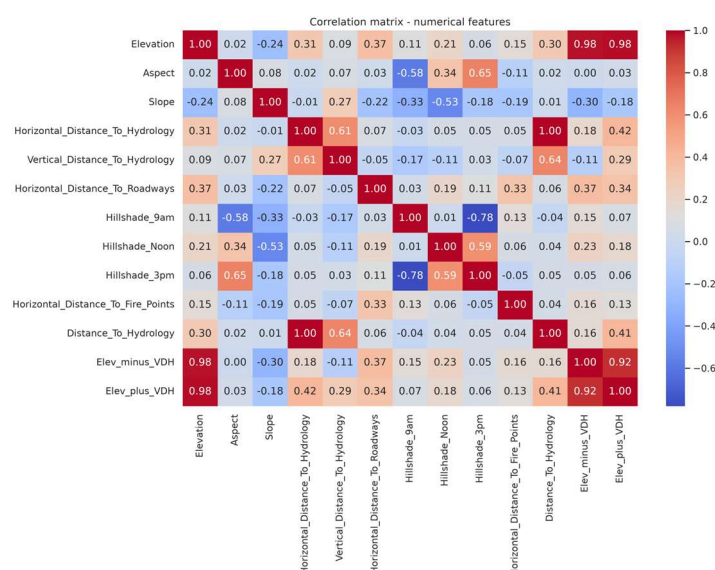
Soil_Type — soil type The log-scale distribution plot shows strong imbalance: Soil Type 28 is the most frequent in the observations, while others are almost absent (such as ID 14). The feature is highly discriminative once the associations between certain species and certain soil types are observed. The heatmap shows that class (7) has a very strong association with soils 38 and 39; class (6) is strongly associated with soil ID 9. Classes (1) and (2) are spread over a wider range of soils (21–32).



2.4 Correlation Matrix

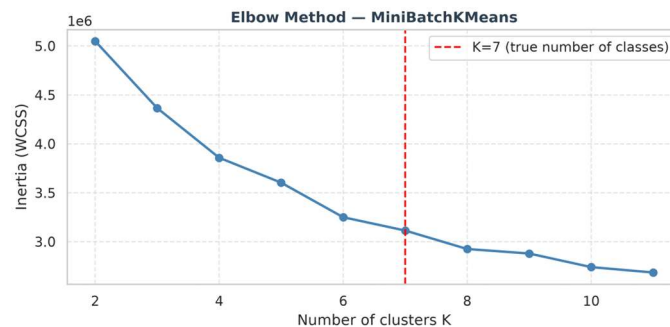
Most relevant correlations:

- **Elevation ↔ Elev_minus_VDH / Elev_plus_VDH ($r \approx 0.98$):** the two engineered features derive from elevation, so this correlation was expected.
- **Hillshade_9am ↔ Hillshade_3pm ($r = -0.78$):** strongly negatively correlated because of the asymmetry between morning and afternoon solar exposure.
- **Horiz. Distance to Hydrology ↔ Distance_To_Hydrology ($r \approx 1.00$):** the engineered feature is strongly correlated with the horizontal component, since the vertical component is very small.
- **Aspect ↔ Hillshade_3pm ($r = 0.65$):** positively correlated, since slope orientation directly conditions the hillshade.



2.5 Clustering and PCA Analysis

A clustering analysis was carried out for exploratory purposes. Because of the size of the dataset, the classic K-Means algorithm would have been computationally expensive. MiniBatchKMeans was therefore used, which reduces computation times by updating the centroids on batches of observations at each iteration.

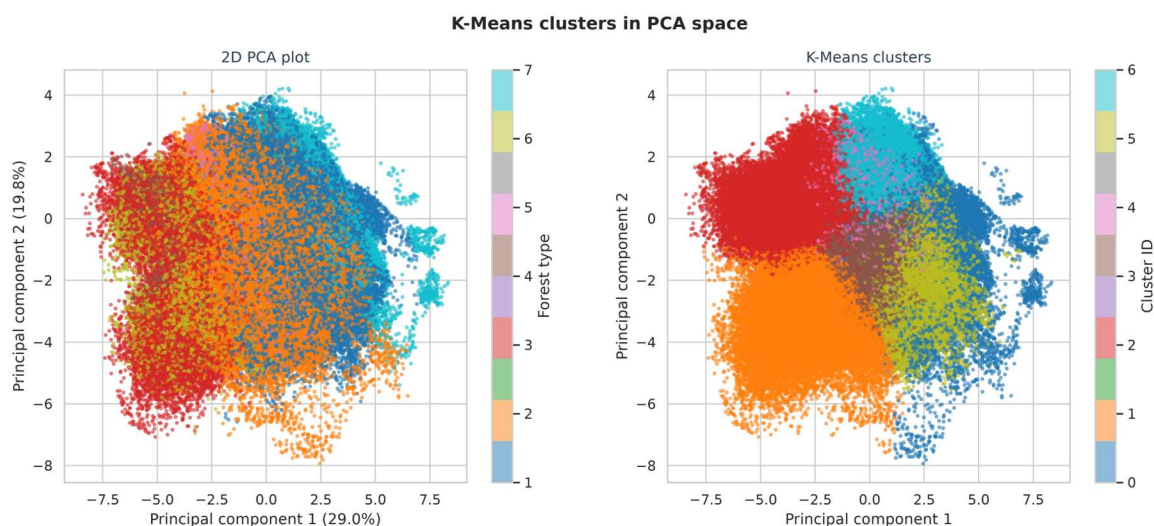


To determine the number of clusters K, the elbow method was used, tracking the within-cluster variation as K varied between 2 and 11. Since the analysis is purely exploratory, K=7 was fixed, also because it is the number of forest species to predict. The resulting clusters were evaluated with two metrics:

- **Adjusted Rand Index (ARI) = 0.0476:** The generated clusters do not align with the true classes. Rather than forming compact regions in the feature space, the species occupy complex, overlapping zones.
- **Silhouette Score = 0.1621:** The clusters are poorly separated and partially overlapping, further reflecting the inherent difficulty in separating the classes.

To compare the real class distribution with the groupings created by the K-Means algorithm, an analysis of the PCA loadings was carried out. The first component captures 29% of the variance of the numerical features and is composed of the three elevation features ($\text{Elev_plus_VDH} \approx 0.48$, $\text{Elevation} \approx 0.47$, $\text{Elev_minus_VDH} \approx 0.43$). The second adds another 19.8%.

Nevertheless, the true classes spread along the PC1 axis with significant overlap, which accounts for the low ARI score. While Elevation provides the strongest signal, it is insufficient on its own. Therefore, accurately separating the classes requires non-linear models.



3 — Preprocessing

3.1 Dataset Split

To prevent data leakage and preserve the class proportions, the dataset was split in a stratified way into a train set (80%) and a test set (20%), yielding 464,809 and 116,203 observations. To reduce computational complexity and computation times, the hyperparameter search was carried out on a reduced, stratified tuning subset (`X_train_tune`) of 100,000 samples. The best hyperparameters were then used to train each model on the full train set of 464,809 observations. For the SVM, because of its $O(n^2)$ computational complexity, training was limited to a subset of 100,000 samples (`X_train_svm`).

3.2 Feature Scaling

Feature transformations followed a different approach depending on the model family, via a `ColumnTransformer`:

- **Linear and distance-based models (LR, KNN, SVM, MLP, LDA/QDA):** Because these models are sensitive to feature scale, the 13 numerical features were standardized using `StandardScaler`, while the categorical ones were encoded with `OneHotEncoder`
- **Tree-based models:** these models are based on comparisons and are therefore immune to scale issues. They received the features directly, without transformations.

3.3 Handling Class Imbalance

Two approaches were used to handle the imbalance:

- **class_weight:** testing different weights as hyperparameters on the Random Forest and LightGBM models.
- **SMOTE:** used to balance the classes for the MLP model.

The other models (Logistic Regression, SVM, Decision Tree, LDA/QDA, etc.) were trained without balancing. As the results show, the most effective method for the minority classes was training on the full dataset, which raised the recall of class 4 from 0.61 to 0.88 and of class 5 from 0.43 to 0.88 in the winning model.

3.4 Memory Optimisation

The Covertypes dataset is large, and to manage execution and training, especially in the cloud, optimisations were needed to reduce RAM usage:

- **Data-type downcasting:** the continuous features, allocated as `float64`, were converted to `float32`. This reduced the dataset's memory footprint, avoiding crashes from RAM saturation or disk errors.
- **Residual management:** at the end of each model's run, the `evaluate_model` function invokes Python's garbage collector and closes all open matplotlib figures.

4 — Technical Choices

To standardise and automate the comparison across all models and to save the results, the following solutions were implemented:

- **Evaluation pipeline:** an `evaluate_model` procedure was created to centralise the metric computation for each model. The procedure computes the main metrics, generates the classification report and saves the confusion matrix as a PNG for each model.
- **Logging:** to avoid losing information during training and to track the results, the standard `print()` function was overridden by a logger, used to duplicate every text output both to screen and to a log file (`run_log.txt`).
- **Figures:** each figure generated within the notebook is automatically saved to the `figures/` folder.

5 — Metrics and Evaluation

The weighted F1-score is the main metric, because it weights each class by its support and therefore reflects performance on a test set that preserves the imbalance of the data. The macro F1-score is reported beside it in every table, because it weights all seven classes equally and falls as soon as a model performs poorly on the rare ones. The distance between the two measures how much of a model's score rests on the majority classes. Accuracy is also reported, for comparison with the literature.

Aggregate metrics do not show which classes a model has failed on, so every model also receives a classification report with per-class precision, recall and F1, and a row-normalised confusion matrix. The same per-class information is collected in two heatmaps that compare all models class by class. The macro OvR AUC and the corresponding ROC curves are computed for every model from which `predict_proba` can be extracted, and one-vs-rest precision-recall curves are produced for five representative models. All figures are saved to the `figures/` folder. The metrics are computed by the `evaluate_model` function on the full test set, for each model refit on the training set with its best hyperparameters.

6 — Model Development and Training

Sixteen classification models were trained, and for each one the hyperparameter search was carried out with `HalvingGridSearchCV` — `HalvingRandomSearchCV` for `LightGBM` — on the reduced, stratified tuning subset of 100,000 observations (`X_train_tune`), using the weighted F1 as the metric because of the class imbalance. The best hyperparameters were then used for training on the full train set of 464,809 observations. Each model is wrapped in a pipeline that applies the `ColumnTransformer` according to the model type. Some models instead follow a different pipeline due to specific needs: the parametric models (`LDA`, `QDA`, `Naive Bayes`) have no significant hyperparameters and were evaluated directly with `cross_val_score` (`cv=5`). The `SVM` is trained on a dedicated subset of 100,000 samples (`X_train_svm`) because of the $O(n^2)$ complexity; the `MLP` uses `SMOTE` in its pipeline to handle the minority classes.

Each trained model was evaluated on the full test set (116,203 observations) with the `evaluate_model` function, which computes the main metrics, generates the per-class classification report and saves the confusion matrix. The ROC curves (one-vs-rest) and the macro AUC were computed for all models from which `predict_proba` could be extracted, except the SVM, which had `probability=False` to contain training times.

6.1 Linear and Parametric Models

All models in this family use the `preprocessor_linear` scaler and receive the 13 numerical features standardised with `StandardScaler` and the 2 categorical ones with `OneHotEncoder`.

Logistic Regression (L1 vs L2): a "classic" Logistic Regression, using the `saga` solver (`max_iter=500`), the only scikit-learn solver that supports both L1 and L2 penalties on multiclass tasks. The hyperparameters tested are:

- $C \in \{0.1, 1, 10\}$ — regularisation strength.
- `penalty` $\in \{L1, L2\}$ — penalty type.

Logistic Regression + PCA: a Logistic Regression variant with dimensionality reduction. Before the classifier, a PCA with `n_components=0.95` is applied, which keeps 95% of the variance and automatically selects the first 12 components. The default (L2) regularisation is tested with $C \in \{0.1, 1, 10\}$, testing different strengths. The aim was to assess whether PCA improved separability, but the results show instead an information loss on the minority classes.

Logistic Regression with Splines. To give the model the ability to capture non-linear relationships, the 13 numerical features pass through a `cubic SplineTransformer` (`degree=3`). The hyperparameters tested were:

- `n_knots` $\in \{3, 4, 5, 6\}$ — control the flexibility of the splines.
- $C \in \{0.1, 1, 10, 100\}$ — regularisation strength (L2 by default).

LDA: given the simplicity of the model, default parameters are used together with classic cross-validation (`cv=5`).

QDA: in this variant it is possible to test regularisation with `reg_param=0.5`, which stabilises the inversion of the covariance matrices. Here too, cross-validation (`cv=5`) is used.

Naïve Bayes: the model is simple, and the analysis is limited to observing its behaviour with default parameters. NB assumes conditional independence and Gaussian distributions for each feature given the class, but this assumption is strongly violated in this dataset, with geographic features that are even highly correlated. This model is probably the least suited to this dataset, and indeed it gave the worst performance in the comparison. A simple cross-validation (`cv=5`) is used here as well.

6.2 Distance- and Margin-Based Models

KNN: The `n_neighbors` hyperparameter was tuned over the set $\{1, 3, 4, 5, 6, 7, 11\}$. The default Euclidean metric from scikit-learn was applied to the output of the `reprocessor_linear` pipeline. Feature standardisation is crucial, because without it raw features with large ranges, such as Elevation (spanning roughly 2,000 metres), would disproportionately dominate smaller-scale features like Slope (measured in degrees). No alternative distance metrics were evaluated, and the use of one-hot encoding ensures that any two distinct soil types remain equidistant in the feature space.

SVM with RBF kernel: the model was trained on the dedicated subset `X_train_svm` of 100,000 samples because of the $O(n^2)$ complexity, where both the hyperparameter search and the refit take place. It was necessary to set `probability=False` and disable probability estimation to contain computation times, so the ROC curve could not be computed. The `kernel='rbf'` was chosen because of the non-linear nature of the dataset, which allows radial decision boundaries to be built. The hyperparameters tested are:

- `C ∈ {1, 10}` — controls the tolerance for misclassified points.
- `gamma ∈ {scale, auto}` — controls the kernel width.

6.3 Decision Trees and Ensembles

The models in this section receive the features without scaling through `preprocessor_trees`, with the exception of Decision Tree, Bagging and AdaBoost, which use `preprocessor_linear` because their scikit-learn implementations do not handle native categorical features.

Decision Tree (Pruned): a single decision tree, optimised through pruning to avoid overfitting. Unlike the other trees, this model receives the `preprocessor_linear` scaler, since scikit-learn does not handle categorical features in classic trees. Combining the depth limit and pruning yields a single, fairly accurate tree. The hyperparameters tested are:

- `max_depth ∈ {10, 20, None}` — maximum tree depth.
- `criterion ∈ {gini, entropy}` — impurity metric for the split.
- `ccp_alpha ∈ {0.0, 0.0001, 0.001}` — aggressiveness of the cost-complexity pruning.

Random Forest: given the class imbalance, two dictionaries were built for this model, starting from the balanced weights and multiplying them by 2 and by 3 on the minority classes to penalise errors. The hyperparameters tested are:

- `n_estimators ∈ {50, 100, 150}` — maximum number of trees in the ensemble.
- `max_depth ∈ {3, 5, 7, None}` — maximum depth of the individual trees.
- `class_weight ∈ {balanced, balanced_subsample, dict_custom_x2, dict_custom_x3}`.

Bagging Classifier: the default bagging also does not support categorical classes, so it uses the `preprocessor_linear` scaler. The hyperparameters tested are:

- `n_estimators ∈ {50, 100, 150}` — maximum number of trees in the ensemble.
- `max_samples ∈ {0.5, 0.8, 1.0}` — % of bootstrapped samples per tree.

Gradient Boosting: the hyperparameters tested are:

- `learning_rate ∈ {0.01, 0.1}` — learning rate.
- `n_estimators ∈ {50, 100, 150}` — maximum number of trees in the ensemble.
- `max_depth ∈ {3, 5, 7}` — maximum depth of the individual trees.

XGBoost: `XGBClassifier` is wrapped in a `CustomXGB` class that uses `LabelEncoder` to map the 1–7 class labels. The model is trained with `tree_method='hist'`, a CPU-run algorithm chosen to avoid GPU memory errors. For a direct comparison, the same parameters as Gradient Boosting are tested.

LightGBM: many hyperparameters can be tested for this model, but computation times must also be balanced. For this reason `HalvingRandomSearchCV` is used, testing these hyperparameters:

- `learning_rate` $\sim U(0.01, 0.21)$ — contribution of each new tree.
- `n_estimators` $\sim \text{randint}(50, 200)$ — trees in the ensemble.
- `max_depth` $\sim \text{randint}(3, 10)$ — maximum depth of the individual tree.
- `min_child_samples` $\sim \text{randint}(10, 100)$ — regularisation on the leaves.
- `subsample` $\sim U(0.5, 0.9)$ — % of rows sampled per tree.
- `colsample_bytree` $\sim U(0.5, 0.9)$ — % of features used per tree.
- `class_weight` $\in \{\text{balanced}, \text{None}, \text{dict_custom_x2}\}$ — penalisation strategies.

AdaBoost: the following hyperparameters were tested:

- `n_estimators` $\in \{50, 100, 150\}$ — number of stumps.
- `learning_rate` $\in \{0.1, 0.5, 1.0\}$ — contribution of each stump.

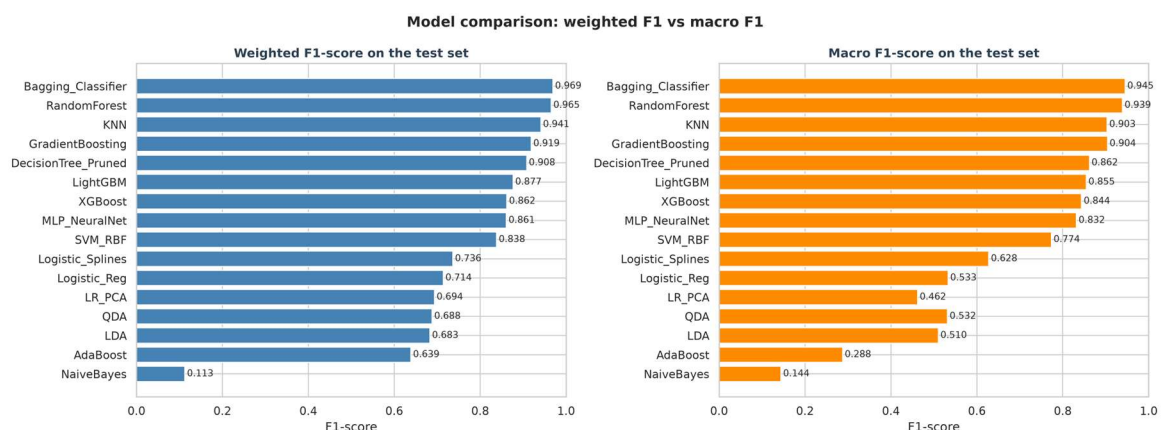
6.4 Neural Network

`MLPClassifier` uses the adam optimiser with `max_iter=1000` and `early_stopping=True`. For the minority classes an `ImbPipeline` is used to apply SMOTE. An AdaptiveSMOTE variant was also implemented, which reduces `k_neighbors` when the smallest class has fewer than 6 samples, avoiding crashes. This proved necessary during local development, since the full dataset could not be used. To contain times, the hyperparameters are selected on 50,000 observations:

- `hidden_layer_sizes` $\in \{(64,), (32, 16), (64, 32, 16)\}$ — hidden-layer architectures.
- `alpha` $\in \{0.0001, 0.05\}$ — weight of the L2 regularisation.

7 — Model Comparison

All 16 models were trained, tuned and evaluated on the same test set of 116,203 observations. The table below summarises the results, sorted by descending test-set F1-score, and the figure compares the weighted F1 with the macro F1.



Model	F1 CV	F1 Test	F1 Macro	Accuracy	AUC-macro
Bagging Classifier	0.9190	0.9693	0.9452	0.9693	0.9986
Random Forest	0.9028	0.9654	0.9389	0.9655	0.9988
KNN	0.8911	0.9413	0.9032	0.9413	0.9440
Gradient Boosting	0.8864	0.9187	0.9040	0.9189	0.9908
Decision Tree Pruned	0.8546	0.9083	0.8620	0.9092	0.9578
LightGBM	0.8701	0.8766	0.8548	0.8749	0.9890
XGBoost	0.8482	0.8622	0.8437	0.8630	0.9854
MLP NeuralNet	0.8213	0.8610	0.8318	0.8594	0.9851
SVM RBF	0.8328	0.8383	0.7736	0.8406	N/A
Logistic Splines	0.7378	0.7364	0.6278	0.7415	0.9483
Logistic Reg	0.7181	0.7142	0.5330	0.7237	0.9362
LR + PCA	0.6970	0.6937	0.4620	0.7062	0.9244
QDA	0.6874	0.6882	0.5317	0.6905	0.9237
LDA	0.6837	0.6830	0.5104	0.6800	0.9019
AdaBoost	0.6425	0.6388	0.2876	0.6688	0.8804
Naive Bayes	0.1121	0.1127	0.1441	0.1222	0.8112

7.1 Reading the Results by Family

The comparison shows a hierarchy among the model families.

The bagging-based tree ensembles (Bagging Classifier and Random Forest) take the top two positions with $F1 \approx 0.97$, followed by KNN (0.941) and the boosting models (Gradient Boosting 0.919, LightGBM 0.877, XGBoost 0.862). The single pruned tree reaches 0.908 while the MLP (0.861) and the SVM RBF (0.838) sit in the middle band.

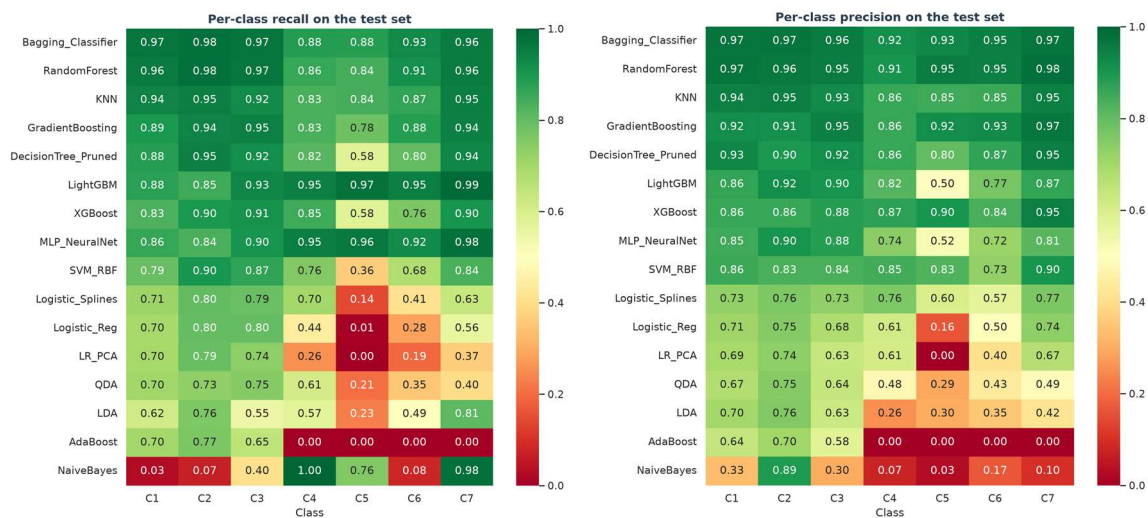
The linear and parametric models (Logistic Regression and variants, LDA, QDA) stop between 0.68 and 0.74, while AdaBoost (0.639) and above all Naive Bayes (0.113) sit at the bottom of the ranking.

The dataset contains continuous geographic variables such as elevation, slope and distances, which generate non-linear decision boundaries. The linear models therefore cannot access part of the information that not even a large amount of data can provide, while the spatial models and the tree ensembles can overcome this limit.

The ordering also depends on which aggregate is read. Random Forest has the highest macro-AUC of all sixteen models (0.9988, above the winner's 0.9986) while sitting second on weighted F1, and Gradient Boosting is below KNN on weighted F1 (0.9187 against 0.9413) but above it on macro F1 (0.9040 against 0.9032).

7.2 Per-Class Behaviour Across Models

The two heatmaps below report recall and precision per class for all 16 models, with the rows in leaderboard order.



Three patterns stand out:

- AdaBoost achieves an accuracy of 0.6688 and a macro-AUC of 0.8804. While these metrics appear reasonable in isolation, its recall on classes 4, 5, 6, and 7 is exactly 0.00, meaning the model exclusively predicts the three most frequent classes.
- LR + PCA exhibits a similar collapse on class 5 (recall 0.00), with standard Logistic Regression performing only marginally better (recall 0.01). This severe failure on minority classes distinguishes them from Logistic Splines, despite sharing similar weighted F1-scores.
- At the opposite extreme, Naive Bayes reaches recall 1.00 on class 4 with precision 0.07. By overpredicting the rarest class it yields the lowest weighted F1-score overall (0.1127), despite a comparatively higher macro F1-score (0.1441).

7.3 The Contribution of the Data

Comparing the performance of a local run on a reduced subset of 25,000 observations against the one obtained on the full dataset shows that, while the linear models do not improve beyond a certain point regardless of the amount of data (or even worsen because of noise), the spatial models and the tree ensembles benefit from more data. While the performance gain was anticipated following the transition to a larger dataset, the distinct behaviours validate the conclusions.

Model	F1 25k	F1 Full (464k)	Delta
KNN	0.820	0.941	+0.121
Bagging	0.862	0.969	+0.107
Decision Tree	0.752	0.908	+0.156
Random Forest	0.844	0.965	+0.121
Logistic Splines	0.736	0.736	0.000
LDA	0.690	0.683	-0.007
AdaBoost	0.643	0.638	-0.005

8 — Winning Model — Bagging Classifier

The Bagging Classifier proved to be the most stable and accurate model across all main metrics. Its performance is summarised below:

Metric	Value
Weighted F1-Score (Test)	0.9693
Macro F1-Score (Test)	0.9452
Accuracy (Test)	0.9693
AUC-macro OvR	0.9986
Mean Bootstrap F1	0.9692
95% Bootstrap CI	[0.9682, 0.9702]
Test set size	116,203 instances

The model was able to recognise in a stable way even the minority classes that were the most critical point of the project. The classification report follows:

Class	Precision	Recall	F1-Score	Support
1 — Spruce/Fir	0.97	0.97	0.97	42,368
2 — Lodgepole Pine	0.97	0.98	0.97	56,661
3 — Ponderosa Pine	0.96	0.97	0.96	7,151
4 — Cottonwood/Willow	0.92	0.88	0.90	549
5 — Aspen	0.93	0.88	0.90	1,899
6 — Douglas Fir	0.95	0.93	0.94	3,473
7 — Krummholz	0.97	0.96	0.97	4,102
Weighted avg	0.97	0.97	0.97	116,203

The majority classes 1 and 2 reach $F1 = 0.97$, but the minority classes 4 and 5, despite their rarity, are also classified effectively.

To rule out variance due to lucky splits, a Bootstrap validation was run on the test set. 1,000 samples were drawn with replacement and the F1-score computed on each, producing a 95% confidence interval. A mean F1 of 0.9692 was obtained, with a confidence interval of [0.9682, 0.9702].

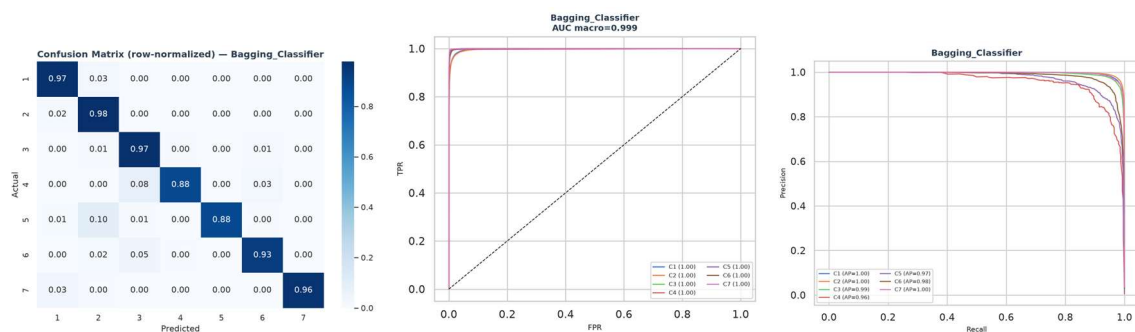
9 — Per-Model Detail

For each of the 16 models, the confusion matrix and the One-vs-Rest ROC curve are placed side by side, with one curve per each of the 7 classes, and for the top five models the precision-recall curves as well, with the Average Precision of each class. The confusion matrices are row-normalised. Given the highly imbalanced class supports (ranging from 56,661 down to just 549 observations), relying on raw counts would obscure misclassification patterns in the minority classes. The results are sorted by test-set F1-score and analysed. Note that the SVM RBF ROC curve is not available for reasons of computational efficiency.

Given this severe class imbalance, the ROC curve is an overly optimistic metric. All 15 applicable models achieved a macro-AUC ≥ 0.8112 , including AdaBoost, despite its complete failure to predict four of the seven classes. Consequently, Precision-Recall (PR) curves were computed for the top five performing models.

1 — Bagging Classifier

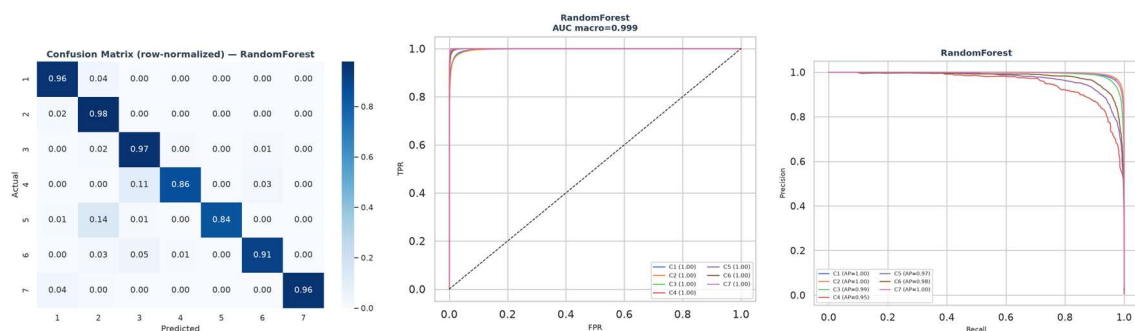
F1 CV = 0.919 | F1 Test = 0.969 | F1 Macro = 0.945 | Accuracy = 0.969 | AUC-macro = 0.999



The matrix is almost diagonal, with errors concentrated almost entirely on the boundary between classes 1 and 2. The minority classes 4 and 5 have very high recall, and all 7 ROC curves coincide with AUC near 1.00. Unlike the Random Forest, Bagging does not introduce random feature selection, letting each tree use all 15 features; the individual trees are therefore stronger and more correlated with each other, but different enough thanks to the bootstrap sampling.

2 — Random Forest

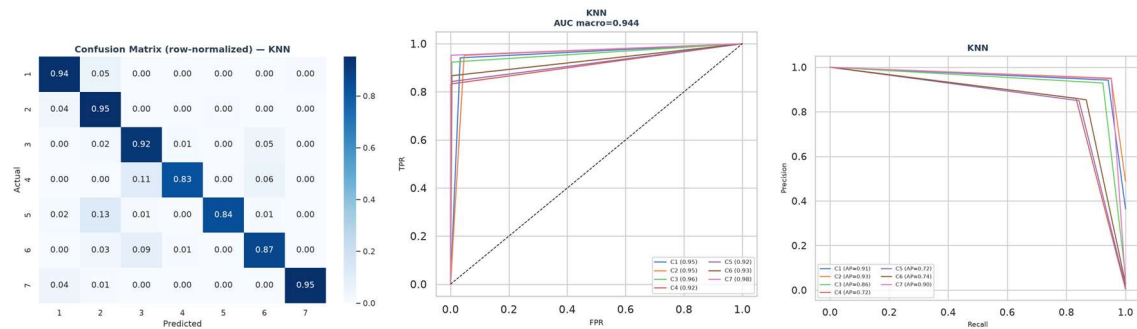
F1 CV = 0.903 | F1 Test = 0.965 | F1 Macro = 0.939 | Accuracy = 0.966 | AUC-macro = 0.999



A pattern almost identical to Bagging, with a strong diagonal and few confusions. Classes 4 and 5 have slightly lower recall than Bagging, probably due to the random feature selection. AUC-macro = 0.999 as in Bagging.

3 — KNN

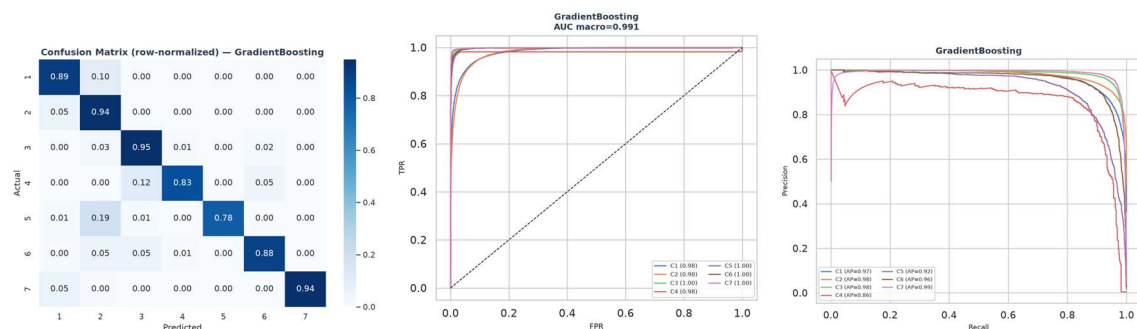
F1 CV = 0.891 | F1 Test = 0.941 | F1 Macro = 0.903 | Accuracy = 0.941 | AUC-macro = 0.944



KNN with K=1 reaches F1 = 0.941 on the full dataset, because forest cover contains continuous geographic variables, so every point in the test set probably has a neighbour in the training set of the same class. The main confusions are between 1 and 2, with classes 5 and 6 remaining the hardest to classify. AUC-macro = 0.944, the lowest among the top 5 models, with stepped curves.

4 — Gradient Boosting

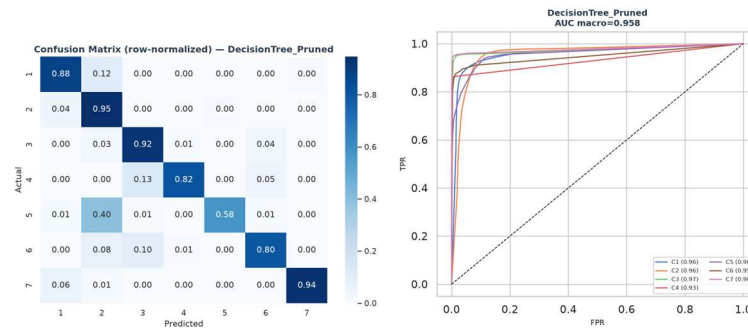
F1 CV = 0.886 | F1 Test = 0.919 | F1 Macro = 0.904 | Accuracy = 0.919 | AUC-macro = 0.991



The model's misclassifications are concentrated in classes 1 and 2. The minority classes 4 and 5 are the hardest here too, but compared with the single pruned tree, boosting reduces the errors on the minority classes. AUC-macro = 0.991 is very high, and the curves for classes 1 and 2 show a slight dip indicating the difficulty of separating the two majority classes. Classes 4–7 instead reach AUC $\approx$ 1.00.

5 — Decision Tree Pruned

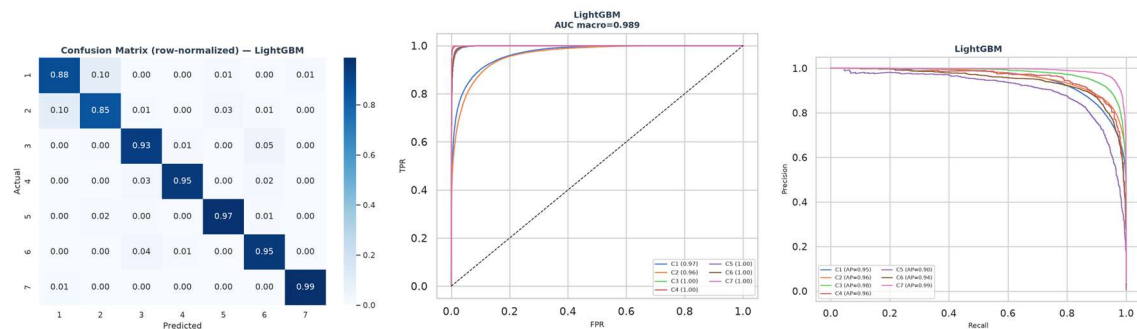
F1 CV = 0.855 | F1 Test = 0.908 | F1 Macro = 0.862 | Accuracy = 0.909 | AUC-macro = 0.958



The single tree reaches $F1 = 0.908$ despite being a simple model. The search for the optimal depth and `ccp_alpha` for the cost-complexity pruning avoided overfitting. With many training samples, each leaf can hold enough observations to form precise conditions. The confusions on classes 1 and 2 are more frequent than in the ensembles. Class 5 with recall 0.58 is the most penalised. AUC-macro = 0.958 and the curves have a more jagged shape than the ensembles.

6 — LightGBM

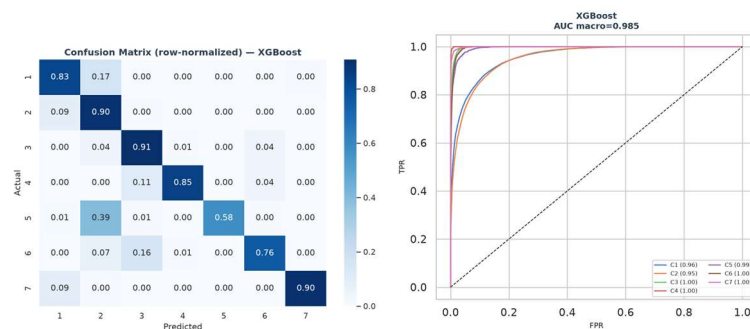
F1 CV = 0.870 | F1 Test = 0.877 | F1 Macro = 0.855 | Accuracy = 0.875 | AUC-macro = 0.989



LightGBM yields lower-than-expected performance. The confusion matrix reveals a significant overprediction of class 5. Despite a high recall (0.97), precision is merely ~50% with false positives predominantly drawn from class 2. This behaviour is a deliberate trade-off. Hyperparameter tuning optimised class weights to boost minority recall at the expense of majority-class false positives. Consequently, the weighted F1-score decreases (0.8766), aligning more closely with the macro F1-score (0.8548). Ranking fourth with a macro-AUC of 0.989, the model's curves highlight its specific struggle in predicting classes 1 and 2.

7 — XGBoost

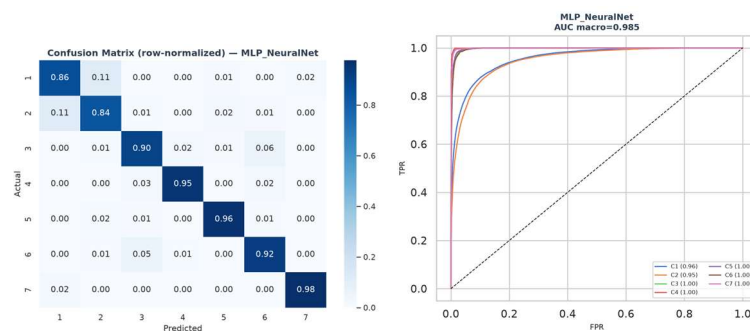
F1 CV = 0.848 | F1 Test = 0.862 | F1 Macro = 0.844 | Accuracy = 0.863 | AUC-macro = 0.985



XGBoost shows the classic difficulty on classes 1 and 2. Class 5 is the most penalised, consistent with the maximum depth explored by the grid ($\text{max_depth} \leq 7$). AUC-macro = 0.985 with curves very similar to LightGBM's and classes 1 and 2 hard to separate.

8 — MLP NeuralNet

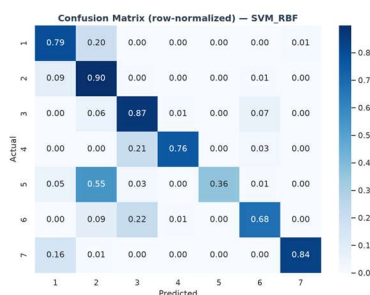
F1 CV = 0.821 | F1 Test = 0.861 | F1 Macro = 0.832 | Accuracy = 0.859 | AUC-macro = 0.985



The MLP does well on the minority classes (4 recall 0.95, 5 recall 0.96) but suffers more on the majority classes. The neural network learned non-linear boundaries for the minority classes better than the ensemble methods, but at the cost of overall precision. AUC-macro = 0.985 and the ROC curves are almost identical to XGBoost's.

9 — SVM RBF

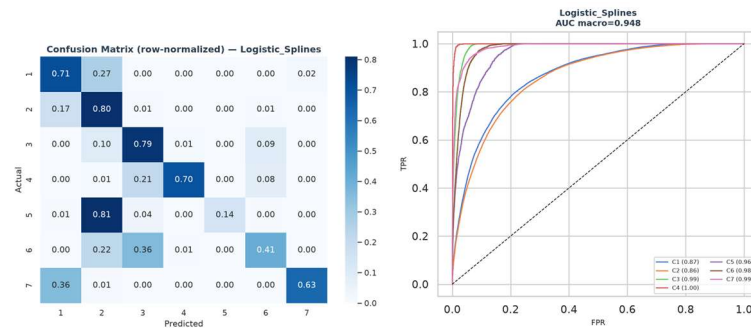
F1 CV = 0.833 | F1 Test = 0.838 | F1 Macro = 0.774 | Accuracy = 0.841 | AUC-macro: N/A



The SVM was trained on a subset of 100,000 samples because of the algorithm's $O(n^2)$ complexity, yet it generalises well to the 116,203 test instances. The RBF kernel maps the non-linear boundaries clearly better than the linear models. Class 5 is the most penalised (recall 0.36), since the SVM saw few examples with the subset. Class 7 shows many false positives towards class 1.

10 — Logistic Splines

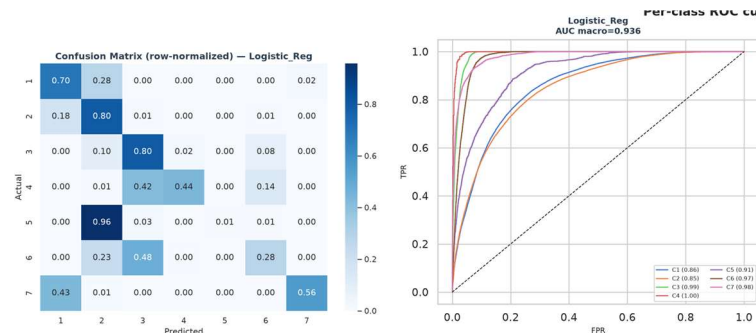
$F1\ CV = 0.738$ | $F1\ Test = 0.736$ | $F1\ Macro = 0.628$ | $Accuracy = 0.742$ | $AUC\text{-}macro = 0.948$



Among the logistic regressions, the use of cubic splines made it possible to capture non-linear relationships. The errors between classes 1 and 2 remain high, and class 5 (recall 0.14) is almost entirely misclassified. AUC-macro = 0.948 is very high, and the gap between AUC = 0.948 and F1 = 0.736 shows good discrimination but imprecise classification thresholds.

11 — Logistic Reg

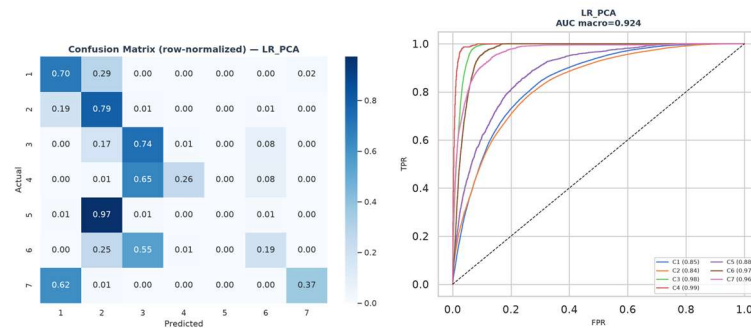
$F1\ CV = 0.718$ | $F1\ Test = 0.714$ | $F1\ Macro = 0.533$ | $Accuracy = 0.724$ | $AUC\text{-}macro = 0.936$



Logistic regression with L1 regularisation reaches F1 = 0.714. Class 5 (recall 0.01) is practically ignored, and the model predicts almost everything as class 1 or 2. Class 7 is often confused with class 1. AUC-macro = 0.936 with the C1 and C2 curves far from the diagonal.

12 — LR + PCA

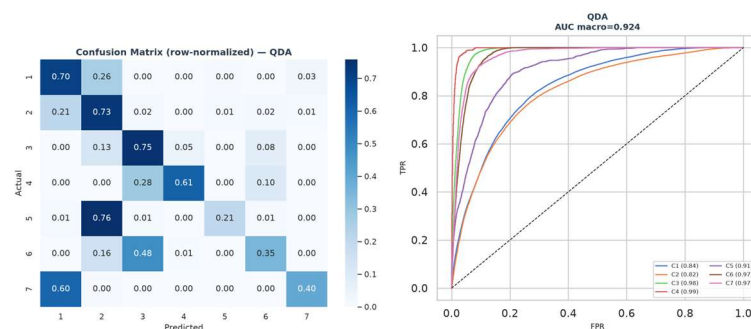
F1 CV = 0.697 | F1 Test = 0.694 | F1 Macro = 0.462 | Accuracy = 0.706 | AUC-macro = 0.924



The PCA variant reaches $F1 = 0.694$, so the PCA preprocessing causes an information loss. The reduction to 12 components (95.3% of variance) removes the information that distinguishes the minority classes. AUC-macro = 0.924 is the worst among the linear regressions. Class 4 reaches AUC ≈ 1.00 despite its low recall.

13 — QDA

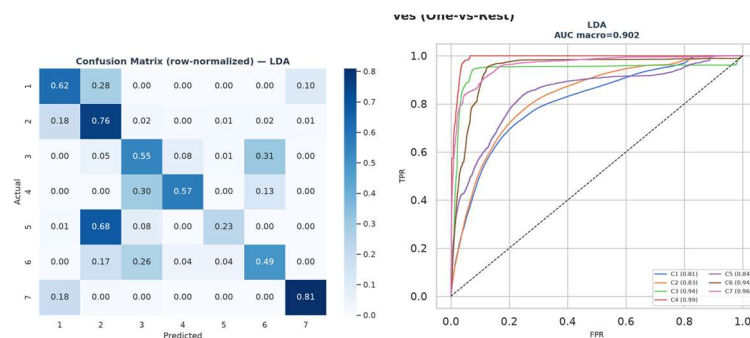
F1 CV = 0.687 | F1 Test = 0.688 | F1 Macro = 0.532 | Accuracy = 0.691 | AUC-macro = 0.924



QDA shows boundaries that improve slightly over LDA. Class 4 (recall 0.61) benefits from the curved boundaries. Classes 5 (recall 0.21) and 6 (recall 0.35) remain problematic. AUC-macro = 0.924, and the C1 and C2 curves show the model's difficulty in separating the two majority classes with parametric methods, while C4 (0.99) has a very high AUC.

14 — LDA

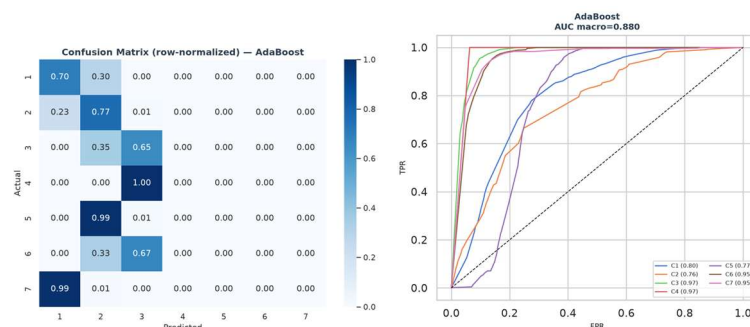
$F1\ CV = 0.684$ | $F1\ Test = 0.683$ | $F1\ Macro = 0.510$ | $Accuracy = 0.680$ | $AUC\text{-}macro = 0.902$



The LDA matrix shows many false positives from class 1 towards class 7. LDA tends to confuse the classes at the extremes of elevation, a behaviour typical of linear separation on multimodal distributions. AUC-macro = 0.902, the worst among the parametric models, with very low C1 and C2 curves.

15 — AdaBoost

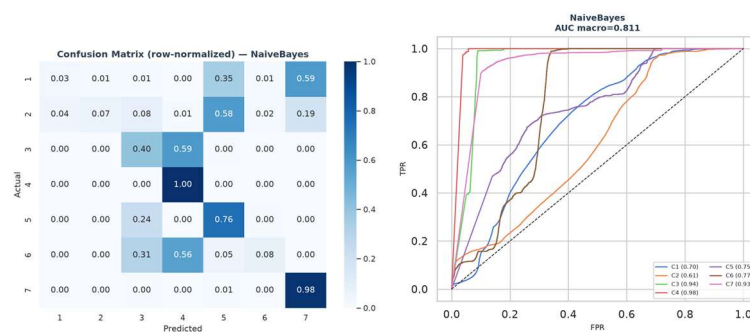
$F1\ CV = 0.643$ | $F1\ Test = 0.639$ | $F1\ Macro = 0.288$ | $Accuracy = 0.669$ | $AUC\text{-}macro = 0.880$



AdaBoost focuses on the misclassified examples and suffers greatly in the presence of noise at the class boundaries, so the ambiguous samples between classes 1 and 2 are constantly revisited and reweighted, ruining performance. The confusion matrix shows that columns 4, 5, 6, 7 are zeroed out (recall = 0.00). The model predicts exclusively classes 1, 2 and 3. AUC-macro = 0.880 with very low curves.

16 — Naive Bayes

$F1\ CV = 0.112$ | $F1\ Test = 0.113$ | $F1\ Macro = 0.144$ | $Accuracy = 0.122$ | $AUC\text{-}macro = 0.811$



Naive Bayes is the least suited model to the problem with $F1 = 0.113$. The independence assumption between features does not hold in a geographic dataset where elevation, slope and hydrology distances are highly correlated. The matrix shows a great many false positives on class 5, and class 4 reaches recall = 1.00 but precision = 0.07. AUC-macro = 0.811 is the worst of all models, and the ROC curves are very unstable.

10 — Limitations

The reported results apply to the process described in Section 3, which has limitations worth stating explicitly. For each limitation, the reason why it was not removed is also provided. Some choices are deliberate, while others are trade-offs due to computational cost.

Spatial Autocorrelation: The Covertypes dataset cells are geographically contiguous and the observations preserve this order. For instance, consecutive rows differ in elevation by 11.65 m on average (compared to 307.55 m for randomly selected rows), and 95.1% of consecutive rows share the same class (versus an expected 37.7% for random ordering). Consequently, a random train-test split distributes adjacent cells across both sets. The most evident symptom is KNN achieving an F1-score of 0.941, indicating that most test cells have an almost identical neighbour in the training set. While a block-based contiguous split would be a more rigorous protocol, adopting it would render these metrics incomparable with the existing Covertypes literature, which relies on random splits.

Tuning Subset: Hyperparameter tuning was performed on a stratified subset of 100,000 observations rather than the full training set, and the SVM was trained exclusively on this subset. The top-performing ensemble models are close to saturation on the hyperparameter grid, therefore extending the search to the entire training set would have drastically increased the computational cost for only marginal performance gains on mid-tier models, without altering the top rankings or the overall conclusions.

Single Split: The bootstrap confidence interval measures test-set sampling noise rather than variance across different data splits. Since cross-validation was not performed on the entire dataset, this interval reflects the precision of the measurement, not the variability of the method itself. A nested cross-validation would multiply the computational load by the number of folds across a 16-model pipeline and would address a different research question. Furthermore, it would not alter the outcome because the gap between the first and second-best models (0.9693 vs. 0.9654) is roughly twice the width of the confidence interval (0.0020).

KNN Quantized Probabilities: With $K = 1$, predicted probabilities are restricted to exactly 0 or 1, resulting in step-wise ROC and Precision-Recall curves for KNN. Its macro-AUC of 0.9440 is therefore not strictly comparable to that of probabilistic models, as it underestimates a classifier that ranks third overall in F1-score (0.9413).

Raw Probabilities: All probability-based analyses utilise the raw outputs returned by the estimators. Brier scores and class-specific decision thresholds were not computed, as model calibration would require re-predicting all models and regenerating all probability-dependent figures. This would not affect the final rankings, which are determined by the F1-score on hard predictions rather than predicted probabilities.

Partial Imbalance Handling: Class weights and SMOTE were evaluated against training on the full dataset, but majority-class undersampling, cost-sensitive thresholds and focal loss were not tested. The conclusion that scaling the dataset aided minority classes more than resampling is therefore strictly bound to the evaluated strategies. Any additional technique would necessitate new tuning and training cycles, falling outside the scope of this project.

Environment-Dependent Results: Running the pipeline on different hardware and a different library stack may cause results to fluctuate at the third or fourth decimal place. This deviation does not affect the rankings or conclusions, and requirements.txt pins the exact versions used to generate the published logs.

None of these limitations calls the comparison itself into question, since they act on the absolute level of the scores and not on their ordering. The one asymmetry is the KNN case noted above.

11 — Conclusions

On this data there is a clear hierarchy between model families. Tree ensembles and distance-based models take the top positions, with the Bagging Classifier first (weighted F1 0.9693) and Random Forest behind it (0.9654), while the linear and parametric models do not exceed 0.74. The reason lies in the nature of the dataset, where elevation, slope and distances produce non-linear decision boundaries.

The most relevant result is that on a dataset where the largest class is 48.76% of the observations and the smallest 0.47%, accuracy and AUC can look solid while the model has stopped predicting four classes out of seven. AdaBoost reaches accuracy 0.6688 and AUC-macro 0.8804 with recall 0.00 on classes 4, 5, 6 and 7. Two aggregate metrics describe it as a working model, but per-class recall shows it has abandoned four of them. For this reason every table in the report places the macro F1 beside the weighted F1, and every model is reported class by class.

For the rare classes, adding more data proved to be the most effective intervention. In a run on the reduced subset, classes 4 and 5 showed a recall of 0.61 and 0.43, respectively, whereas on the full dataset, the Bagging Classifier brought both up to 0.88.

The most concrete contribution of this project is the methodology and pipeline used to build the ranking, featuring sixteen models trained on the exact same split, an isolated test set, and results evaluated on a per-class basis.